

InfraCopilot AI

RAG Chatbot + AI Agent Idea for a Diagram-to-AWS Infrastructure Platform

A production-grade learning and interview project covering indexing, chunking, retrieval, agent orchestration, error handling, monitoring, evaluation, security, and AWS automation.

Project positioning

You already have the strongest base: a visual canvas that can draw infrastructure and create real AWS resources. The AI extension should not be a generic chatbot. It should become a cloud architect, Terraform reviewer, AWS account analyst, runbook assistant, and safe deployment copilot.

Target user	Cloud engineer, DevOps engineer, platform engineer, tech lead, or student learning AWS automation.
Main outcome	User can ask questions, validate a diagram, generate IaC, understand cost/security risk, and safely execute approved AWS actions.
AI concepts covered	RAG, agentic RAG, tool calling, structured outputs, memory, evaluation, tracing, observability, guardrails, multi-tenancy, and HITL approval.
Backend fit	.NET Core or Node.js API with background workers, vector store, document store, event bus, and AWS integrations.
Interview story	This is an advanced real-world AI system, not only a demo. It connects user intent, internal knowledge, cloud telemetry, and safe action execution.

Table of Contents

1. Product idea and feature set
2. High-level architecture
3. Knowledge sources and indexing strategy
4. Chunking strategies for documents, Terraform, logs, and diagrams
5. RAG query pipeline
6. AI agent workflows and tool design
7. AWS execution safety and human approval
8. Error handling, retries, DLQ, and resilience
9. Monitoring, tracing, evaluation, and dashboards
10. Security, governance, and multi-tenancy
11. Implementation roadmap
12. Interview explanation and talking points

1. Product Idea: InfraCopilot AI

Add an AI layer to your existing infrastructure drawing application. The AI should understand the canvas, the generated Terraform/CloudFormation, user-uploaded documents, AWS account telemetry, and deployment history. The result is a RAG chatbot plus a safe AI agent that can reason, recommend, validate, and execute approved actions.

Core modules

- **Diagram-aware RAG chatbot:** Users can ask: "Why did you choose this architecture?", "What is missing for production?", "Explain this VPC design", or "How can I reduce Lambda cost?"
- **Architecture reviewer:** The system checks the diagram against AWS Well-Architected style principles: security, reliability, cost, scalability, and operational excellence.
- **Terraform/IaC assistant:** Generates, explains, reviews, and fixes Terraform modules, provider versions, variables, secrets handling, and outputs.
- **AWS live account analyst:** Uses safe read-only tools to inspect billing, resources, CloudWatch metrics, IAM policies, and drift between canvas and real AWS.
- **Deployment copilot agent:** Plans required actions, asks for approval, triggers provisioning, monitors progress, and reports success/failure with trace IDs.
- **Incident assistant:** Uses logs, metrics, CloudWatch alarms, deployment history, and runbooks to explain failures and suggest fixes.
- **Learning mode:** Turns each user action into interview-style explanations: RAG flow, indexing, chunking, vector search, agent tools, retries, observability, and cloud architecture decisions.

Best interview-friendly project name

InfraCopilot AI: an agentic RAG platform for AWS infrastructure design, validation, deployment, troubleshooting, and cost/security governance.

2. High-Level Architecture

The design below keeps your visual infrastructure builder as the core product and adds an AI platform around it.

Layer	Components	Responsibilities	Interview concepts
Frontend	Diagram canvas, chat panel, approval modal, deployment timeline	User draws infra, asks questions, reviews AI plan, approves actions	UX for HITL, streaming responses, stateful chat
API Layer	.NET/Node APIs, auth, tenant context, rate limits	Receives diagram JSON, chat prompts, document uploads, execution commands	CQRS, MediatR, validation, throttling, JWT/OAuth
RAG Layer	Loaders, chunkers, embedding service, vector DB, metadata DB, reranker	Indexes knowledge and retrieves evidence with citations	Chunking, embeddings, hybrid search, reranking
Agent Layer	Planner, tools, memory, policies, graph	Decides steps, calls tools, handles failures,	LangGraph-style state machine, tool calling,

	workflow	asks approval	durable execution
AWS Tooling	Read-only tools, Terraform runner, CloudWatch, Cost Explorer, IAM analyzer	Inspects and provisions AWS safely	Least privilege, idempotency, drift detection
Observability	OpenTelemetry, LangSmith-style traces, logs, metrics, dashboards	Tracks latency, token cost, retrieval quality, tool failures	Tracing, evaluation, alerting, SLOs
Storage	S3, SQL/Postgres, Redis, vector DB, audit log	Stores docs, chunks, embeddings, sessions, execution history	Multi-tenancy, caching, versioning

Reference flow

1. User draws an AWS architecture: VPC, subnets, Lambda, API Gateway, SQS, RDS, S3, IAM roles.
2. Application stores diagram JSON and generates Terraform modules.
3. Indexing worker converts diagram JSON, Terraform, AWS docs, internal runbooks, and deployment logs into searchable chunks.
4. User asks: "Is this production ready and what will it cost?"
5. RAG retrieves relevant AWS rules, project policies, previous diagrams, Terraform snippets, and cost data.
6. Agent creates a structured plan: missing items, risks, recommended changes, expected cost, and execution steps.
7. Any action that changes AWS requires human approval before execution.
8. After execution, the system monitors CloudWatch, updates deployment history, and indexes the new result for future Q&A.

3. Knowledge Sources and Indexing Strategy

A strong RAG system is not only vector search. It needs clear source ownership, metadata, versioning, indexing pipelines, failure handling, and evaluation.

Source	Examples	Chunking style	Metadata	Use case
Diagram JSON	Nodes, edges, AWS service config	Resource-level chunks	tenantId, diagramId, version, resourceType	Explain architecture and detect missing dependencies
Terraform/HCL	modules, variables, outputs, provider config	Code-aware chunks by block/module	module, resourceType, filePath, gitCommit	Review IaC and generate fixes
AWS docs/runbooks	service limits, deployment rules, standards	Recursive/semantic chunks	source, section, updatedAt, docType	Answer best practice questions
AWS account data	Resources, tags, costs, metrics, IAM policies	Record-level or time-window chunks	accountId, region, service, timestamp	Live cloud analysis and drift detection
Logs/metrics/	CloudWatch logs,	Time-window	traceId,	Troubleshooting

incidents	alarms, deployment failures	chunks with event grouping	deploymentId, severity, service	and root cause analysis
User conversations	Past chat sessions and approvals	Session summary chunks	userId, sessionId, intent, createdAt	Memory and personalization

Indexing pipeline

9. Ingestion trigger: file upload, diagram save, Terraform generation, AWS sync, or deployment completion.
10. Pre-processing: validate file type, extract text/code/JSON, remove secrets, normalize whitespace, detect language.
11. Chunking: use source-specific chunkers instead of one generic chunk size for everything.
12. Metadata enrichment: attach tenant, source, version, diagram id, resource type, AWS account id, region, and sensitivity level.
13. Embedding: call embedding model with batching, retries, rate limit handling, and dead-letter queue for failures.
14. Index write: upsert into vector DB with deterministic IDs so updates are idempotent.
15. Quality check: sample chunks, detect empty chunks, oversize chunks, missing metadata, and duplicate vectors.
16. Versioning: keep old versions for audit but search latest by default unless the user asks historical questions.

4. Chunking Strategies

For interviews, explain that chunking is domain-specific. A fixed 800-character chunk is simple, but production systems need structure-aware chunking.

Content type	Recommended chunking	Why	Example metadata
PDF/docs	Recursive splitting by heading/paragraph with 10-20% overlap	Keeps semantic context and avoids broken sentences	docId, headingPath, pageNumber
Terraform/HCL	Chunk by resource/module/variable block	A resource block should not be split randomly	resourceName, resourceType, moduleName
Diagram JSON	Chunk by node, edge, group, and complete architecture summary	Resource-level retrieval plus full-architecture context	nodeId, connectedTo, serviceType
Logs	Chunk by request trace, time window, or error cluster	Root cause needs event sequence	traceId, service, severity, timestamp
Cost data	Chunk by service, region, account, date range	Cost questions need aggregation context	accountId, service, month
Chat memory	Summarized conversation chunks after every session	Avoids storing too much raw conversation	sessionId, summaryType, userIntent

Advanced chunking idea

Create a Diagram Context Chunk every time the user saves the canvas. It should summarize services, dependencies, security groups, IAM roles, estimated cost drivers, and generated Terraform files. This makes the chatbot deeply aware of the current architecture.

5. RAG Query Pipeline

Use a multi-step RAG pipeline instead of direct prompt + vector search. This gives you advanced talking points for interviews.

17. Intent classification: classify query as explanation, code generation, cost analysis, security review, troubleshooting, deployment action, or learning mode.
18. Query rewrite: rewrite vague user text into precise cloud/search terms. Example: "why lambda slow" becomes "Lambda cold start, memory, timeout, VPC ENI, CloudWatch duration metrics".
19. Retrieval plan: choose collections/namespaces: diagram, Terraform, AWS docs, runbooks, logs, cost, chat memory.
20. Hybrid retrieval: combine vector search for semantic match and keyword/BM25 search for exact service names, error codes, resource IDs, and Terraform resource names.
21. Reranking: rerank retrieved chunks based on query relevance, recency, source authority, tenant match, and current diagram version.
22. Context compression: remove duplicate chunks and summarize long evidence before sending to the LLM.
23. Grounded answer: answer with citations, mention assumptions, and refuse to invent missing facts.
24. Post-answer validation: run a hallucination/grounding check before final response.
25. Feedback loop: capture thumbs up/down, correction, missing source, and create evaluation dataset entries.

Suggested RAG answer contract

Field	Meaning
answer	Final user-friendly answer.
sources	Retrieved chunks with source title, page/section, confidence, and URL/path.
assumptions	Anything inferred from incomplete data.
risks	Security, cost, reliability, or compliance risks.
recommendedActions	Safe next steps; write actions require approval.
confidence	High/medium/low based on retrieval quality and evidence.

6. AI Agent Workflows and Tool Design

The agent should not directly execute AWS changes from a prompt. Design it as a state machine with tool permissions, validation, and approval gates.

Agent/tool	Purpose	Read/write level	Examples
Planner agent	Break user goal into	No AWS write	"Create VPC + Lambda

	steps		API" -> plan and risk list
RAG retriever tool	Fetch evidence	Read only	Search diagrams, Terraform, docs, runbooks
Terraform generator tool	Generate IaC	Local write only	Create module files, variables, outputs
Terraform validator tool	Run fmt/validate/plan	No AWS write until approval	Show planned changes
AWS read tool	Inspect live account	Read only	List Lambda, SQS, RDS, IAM, cost
Deployment tool	Apply approved changes	Write with approval only	Apply Terraform, rollback on failure
Monitoring tool	Check health after deployment	Read only	CloudWatch metrics, logs, alarms
Incident tool	Analyze failures	Read only by default	Group logs, identify likely root cause

Agent state machine

26. Understand user goal and current diagram state.
27. Retrieve relevant knowledge using RAG.
28. Create an execution plan with risks and required approvals.
29. Validate the plan against policy: IAM, cost threshold, environment, region, naming, tags.
30. Ask human approval for any action that creates, updates, or deletes AWS resources.
31. Execute using idempotent tool calls and record trace ID, deployment ID, and step status.
32. Monitor result and generate final summary with logs, metrics, created resources, and rollback option.

Advanced interview concept

Explain the agent as a graph, not a while-loop. Nodes represent classify, retrieve, plan, validate, approve, execute, monitor, and summarize. Edges decide the next step based on state, errors, and policy.

7. AWS Execution Safety and Human Approval

Because your app creates real AWS infrastructure, safety is a core architecture topic. This will impress interviewers because it shows production thinking.

- **Least privilege IAM:** Separate read-only role, plan role, and deploy role. The AI agent should never use admin credentials.
- **Approval gate:** Terraform apply, resource deletion, IAM policy changes, public exposure, and cost-increasing changes must require user confirmation.
- **Policy engine:** Validate tags, region, budget, security group rules, public S3 blocks, encryption, and naming convention before apply.
- **Dry run first:** Always show Terraform plan or change set before executing.
- **Idempotency:** Use deploymentId and deterministic resource names to avoid duplicate resources on retry.
- **Rollback strategy:** Keep previous state, support destroy for failed sandbox deployments, and record rollback commands.

- **Audit log:** Store who approved what, when, prompt, plan, diff, tool calls, and final result.

8. Error Handling, Retries, DLQ, and Resilience

Failure	Detection	Handling	Interview keyword
Empty or corrupt document	Extractor returns empty text	Reject with validation message and store failed ingestion event	Input validation
Chunk too large or too small	Chunk audit finds token overflow/empty chunk	Re-chunk with fallback splitter	Chunk quality checks
Embedding API rate limit	429/timeout	Exponential backoff, queue retry, DLQ after max attempts	Resilience pattern
Vector DB upsert failure	Write exception or dimension mismatch	Validate embedding dimensions, retry idempotently	Idempotency
Poor retrieval quality	Low score/no source diversity	Query rewrite, hybrid search, ask clarification, return low confidence	Grounded RAG
LLM hallucination risk	Answer references facts not in context	Grounding validator, cite-only answers, abstain when needed	Guardrails
Tool execution failure	AWS/Terraform error	Classify retryable vs non-retryable, rollback, human escalation	Saga/compensation
Long-running deployment	Job exceeds API timeout	Background worker, status polling, durable state	Async workflow
Partial AWS creation	Some steps success, some fail	Compensating action and deployment state machine	Saga pattern

Production-grade retry pattern

- Use queue-based ingestion and deployment jobs instead of doing everything inside HTTP request-response.
- Use exponential backoff with jitter for rate limits, network timeouts, and temporary service failures.
- Use dead-letter queues for documents/deployments that repeatedly fail.
- Store retry count, last error, correlation ID, chunk ID, source ID, and tenant ID.
- Make upsert and deployment actions idempotent using deterministic IDs.
- Expose a retry/reprocess button in admin UI for failed ingestion jobs.

9. Monitoring, Tracing, Evaluation, and Dashboards

Monitoring should cover two dimensions: normal software observability and AI quality observability.

Area	Metrics	Why it matters	Tools
API performance	p95 latency, error rate, throughput	Shows system health	OpenTelemetry, Prometheus, Grafana

RAG retrieval	recall@k, precision@k, MRR, source coverage	Shows whether the right chunks are retrieved	LangSmith/RAG evals, custom evaluator
LLM quality	groundedness, faithfulness, answer correctness	Reduces hallucination risk	LLM-as-judge, golden dataset
Agent tools	tool success rate, retries, timeouts, approval rate	Shows agent reliability	Trace spans and dashboards
Cost	token cost, embedding cost, vector DB cost, AWS cost delta	Controls production cost	Cost dashboard and budget alerts
Indexing	documents processed, failed chunks, queue lag	Keeps knowledge base fresh	Worker metrics and DLQ dashboard
Security	blocked actions, policy violations, secret redaction events	Shows governance	Audit log and SIEM

Trace design

- Create one root trace per user request.
- Add child spans for classify, rewrite, retrieve, rerank, LLM call, tool call, approval wait, Terraform plan/apply, and monitoring check.
- Attach attributes: tenantId, userId hash, diagramId, model, promptTokens, completionTokens, vectorTopK, confidence, toolName, awsRegion, deploymentId.
- Never log secrets, full credentials, or sensitive customer data in traces.
- Show trace ID in UI so support engineers can debug exact failures.

Evaluation dataset

- Create 50-100 golden questions from your own app: diagram explanation, Terraform issue, cost question, Lambda troubleshooting, IAM risk, VPC design, deployment failure.
- Store expected answer, required source chunks, allowed assumptions, and pass/fail criteria.
- Run evals after every prompt, retriever, chunking, or model change.
- Track regression over time: answer correctness, citation accuracy, hallucination rate, latency, and cost.

10. Security, Governance, and Multi-Tenancy

- **Tenant isolation:** Every chunk, vector, chat session, tool call, and deployment must carry tenantId/workspaceId. Retrieval must filter by tenant before returning evidence.
- **Data protection:** Mask secrets before indexing. Do not embed access keys, tokens, passwords, private keys, or customer PII.
- **Prompt injection defense:** Treat documents and logs as untrusted data. Retrieved text must not be allowed to override system policies.
- **Tool authorization:** Separate permissions by role: viewer can ask, developer can plan, admin can approve deployment.
- **Auditability:** Save prompt, retrieved sources, model version, tool calls, approvals, execution result, and final answer.

- **Environment separation:** Sandbox/dev/prod should use separate credentials, vector namespaces, and deployment policies.
- **Budget guardrails:** Block or require extra approval when estimated monthly cost crosses configured threshold.

11. Implementation Roadmap

Phase	Build	Advanced AI concepts	Demo outcome
Phase 1	Basic RAG chatbot over AWS docs, Terraform files, and diagram JSON	Indexing, metadata, embeddings, vector search, citations	Ask questions about current diagram and IaC
Phase 2	Diagram-aware architecture reviewer	Query classification, hybrid retrieval, reranking, structured output	Get production-readiness report
Phase 3	Terraform generator/reviewer	Tool calling, code-aware chunking, validation loop	Generate and fix Terraform safely
Phase 4	AWS read-only live account agent	Agent tools, least privilege, drift detection	Compare diagram with real AWS account
Phase 5	Safe deployment agent	Human-in-the-loop, state machine, saga, retries	Plan, approve, apply, monitor
Phase 6	Observability and eval platform	Tracing, metrics, golden dataset, LLM evaluation	Show dashboards and regression tests
Phase 7	Incident/copilot mode	Agentic RAG, memory, log retrieval, root cause summary	Troubleshoot failed deployment

Suggested tech stack

Concern	Option	Why
Backend API	.NET 8 Web API or Node.js/NestJS	Fits your full-stack background
Workers	Hangfire, BullMQ, Azure/AWS queue workers, or background service	Handles ingestion and deployment jobs
Vector DB	Pinecone, Qdrant, Weaviate, Chroma for local	Stores embeddings and metadata filters
Relational DB	PostgreSQL or SQL Server	Stores tenants, diagrams, jobs, audit, approvals
Object store	S3	Stores uploaded docs, Terraform zip, logs export
Agent framework	LangGraph, Semantic Kernel, or custom graph executor	Good for stateful agent workflows
Observability	OpenTelemetry + Prometheus/Grafana + LangSmith-style LLM traces	Covers app + AI monitoring
Cloud automation	Terraform CLI, AWS SDK, CloudWatch, Cost Explorer	Real AWS provisioning and analysis

12. Interview Explanation and Talking Points

One-minute explanation

I built a diagram-to-AWS infrastructure platform where users can visually design cloud resources and provision them on AWS. I extended it with an agentic RAG layer called InfraCopilot AI. The system indexes diagram JSON, Terraform code, AWS docs, internal runbooks, deployment logs, and live AWS metadata. The chatbot answers with citations, while the AI agent can review architecture, generate Terraform, detect drift, estimate cost, troubleshoot deployment failures, and execute approved AWS actions safely. I designed it with source-specific chunking, hybrid retrieval, reranking, structured outputs, human approval, idempotent tool execution, retries, DLQ, OpenTelemetry tracing, LLM evaluation, and audit logging.

Deep-dive answers you can give

Question	Strong answer
How did you design chunking?	I avoided one-size-fits-all chunking. AWS docs use heading/paragraph chunks, Terraform uses resource/module blocks, diagrams use node/edge/resource chunks, and logs use trace/time-window chunks. Every chunk has metadata such as tenantId, source, version, service type, region, and sensitivity.
How do you reduce hallucination?	I use retrieval with citations, source filtering by tenant and version, reranking, context compression, answer confidence, and a post-answer grounding check. If evidence is weak, the system says it is not sure instead of guessing.
How does the agent execute AWS actions safely?	The agent first creates a plan, validates policy, shows Terraform plan, and asks human approval. Only approved actions can use the deploy role. All actions are idempotent and audited.
How do you handle failures?	Indexing and deployment are queue-based. I use retries with exponential backoff, DLQ, deterministic IDs, error classification, rollback/compensation, and visible job status in UI.
How do you monitor the AI system?	I trace every request using OpenTelemetry-style spans for classify, retrieve, rerank, LLM call, tool call, approval, deployment, and monitoring. I track latency, cost, retrieval quality, hallucination rate, tool failures, and user feedback.
How do you evaluate RAG?	I created golden datasets from real use cases. I measure retrieval recall@k, source correctness, groundedness, answer correctness, latency, and token cost after each prompt or retriever change.
Why agentic RAG instead of normal RAG?	Normal RAG retrieves once and answers. Agentic RAG can iteratively search, inspect diagram state, call tools, validate Terraform, check AWS metrics, and reason through a multi-step cloud task with approval gates.

Sample demo script

33. Open your app and draw API Gateway -> Lambda -> SQS -> Worker Lambda -> RDS -> S3.
34. Ask chatbot: "Explain this architecture and tell me production risks."
35. Show citations from AWS docs/runbook and chunks from the current diagram.
36. Ask: "Generate Terraform and validate it." Show Terraform plan only, not apply.
37. Ask: "Compare this with my AWS account." Show read-only drift detection.
38. Approve a sandbox deployment. Show trace, progress, status, and CloudWatch health check.
39. Show monitoring dashboard: token cost, latency, retrieval score, tool success, and deployment trace.

Mini backlog for advanced learning

- Add multimodal diagram understanding: convert canvas screenshot into architecture summary and compare with diagram JSON.
- Add policy-as-code checks using OPA/Rego or custom rules before deployment.
- Add cost simulation: estimate monthly spend before creating resources.
- Add drift auto-remediation suggestions but keep execution approval-based.
- Add memory: summarize user preferences like preferred region, naming convention, budget limit, and default tags.
- Add prompt injection tests: malicious document tells agent to ignore policy; verify tool guardrails block it.
- Add retriever A/B test: compare recursive chunks vs semantic chunks vs code-aware chunks.
- Add model routing: small model for classification, stronger model for planning, embedding model for indexing.

References and inspiration

- **LangGraph documentation:** Stateful agents, durable execution, human-in-the-loop, and memory concepts. <https://docs.langchain.com/oss/python/langgraph/overview>
- **LangSmith documentation:** Tracing, evaluation, prompt testing, and deployment monitoring for LLM apps. <https://docs.langchain.com/langsmith/home>
- **LangSmith RAG evaluation tutorial:** Creating datasets and measuring RAG application performance. <https://docs.langchain.com/langsmith/evaluate-rag-tutorial>
- **Amazon Bedrock Knowledge Bases:** AWS managed RAG capability for retrieving proprietary information. <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html>
- **OpenTelemetry documentation:** Vendor-neutral traces, metrics, and logs for observability. <https://opentelemetry.io/docs/>